ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №9

«Паралельність, багатопроцесність та оптимізація в Python»

з курсу «Професійний Python»
Варіант №3 — «Інтернет-магазин»

Виконав(ла): Вересюк Ірина

Група: ФЕП-32

Перевірив(ла):Куньо І.М.

Дата: 28.10.26

Львів — 2026

1. Мета роботи

Ознайомитися з основними підходами до паралельного та конкурентного виконання програм у Python, дослідити вплив Global Interpreter Lock (GIL) на виконання CPU-bound та I/O-bound задач. Навчитися використовувати threading, multiprocessing, ThreadPoolExecutor та ProcessPoolExecutor. Навчитися виявляти вузькі місця (bottlenecks) за допомогою cProfile та tracemalloc, застосовувати векторизацію обчислень за допомогою NumPy, усувати Race Conditions за допомогою Lock та оптимізувати повторні операції через caching з інвалідацією.

2. Індивідуальний варіант і постановка задачі

Варіант №3 — «Інтернет-магазин». Товар має назву, категорію, ціну та кількість. Зберегти базові можливості: додавання товару, обчислення вартості залишків, пошук, фільтрацію за категорією та визначення найдорожчого товару.

На каталозі від 10 000 до 100 000+ товарів виконати профілювання вартості залишків, агрегованої статистики категорій та масового алгоритму динамічного перерахунку цін (CPU-bound). Порівняти послідовний baseline на Python loops, векторизацію NumPy, ThreadPoolExecutor та ProcessPoolExecutor. Змоделювати конкурентне оновлення залишків на складі в пам'яті, зафіксувати race condition та усунути її через Lock. До статистики категорій застосувати lru_cache із підтримкою cache invalidation.

3. Структура проєкту

```
online_store_manager/
├── pyproject.toml
├── README.md
├── .gitignore
├── report/
│   └── Вересюк_Ірина_Лаб09_V3.pdf
├── src/
│   └── store_manager/
│       ├── __init__.py
│       ├── models.py
│       ├── repository.py
│       ├── analytics.py
│       ├── parallel.py
│       └── profiling.py
├── tests/
│   └── test_store_manager.py
└── benchmarks/
    ├── benchmark.py
```

└── results/

└── benchmark_results.csv

4. Опис створених модулів

4.1. models.py

Для представлення товару використано датаклас Product із прапорцями slots=True та frozen=True для хешованості об'єктів під час кешування. Властивість total_value обчислює сумарну вартість позиції на складі.

```
from dataclasses import dataclass


@dataclass(slots=True, frozen=True)

class Product:

  id: int

  name: str

  category: str

  price: float

  quantity: int


  @property

  def total_value(self) -> float:

    return self.price * self.quantity
```

Модуль відповідає за незмінний шар даних проєкту. Використання frozen=True гарантує безпечне хешування об'єктів у колекціях, а slots=True суттєво оптимізує споживання оперативної пам'яті, запобігаючи створенню динамічних словників __dict__ для кожного екземпляра товару.

4.2. analytics.py

Модуль реалізує базовий послідовний розрахунок залишків через цикли, векторизовану версію на NumPy, CPU-bound алгоритм перерахунку цін heavy_price_adjustment та LRU-кешування аналітики за категоріями з механізмом інвалідації.

```
from __future__ import annotations

import math

from functools import lru_cache

from typing import List, Dict, Any, Tuple

import numpy as np

from .models import Product
```

```
def calculate_total_stock_value_baseline(products: List[Product]) -> float:
    """Baseline: підрахунок загальної вартості через звичайний цикл Python."""
    total = 0.0
    for p in products:
        total += p.price * p.quantity
    return total

def calculate_total_stock_value_numpy(prices: np.ndarray, quantities: np.ndarray) -> float:
    """Векторизований підрахунок вартості складських залишків за допомогою NumPy."""
    return float(np.sum(prices * quantities))

def heavy_price_adjustment(price: float) -> float:
    """CPU-bound функція динамічного ціноутворення."""
    val = price
    for _ in range(80):
        val = math.sqrt(abs(val) + 1.0) + math.sin(val) ** 2 + math.cos(val) ** 2
    return round(val * 1.05, 2)

def reprice_products_baseline(prices: List[float]) -> List[float]:
    """Послідовний перерахунок цін (CPU-bound baseline)."""
    return [heavy_price_adjustment(p) for p in prices]

def calculate_category_stats_baseline(products: List[Product], category: str) -> Dict[str, Any]:
    """Розрахунок агрегованої статистики для категорії без кешування."""
    filtered = [p for p in products if p.category == category]
    if not filtered:
        return {"category": category, "count": 0, "avg_price": 0.0, "total_value": 0.0, "max_price": 0.0}

    prices = [p.price for p in filtered]
    total_val = sum(p.price * p.quantity for p in filtered)
    return {
        "category": category,
```

```
        "count": len(filtered),
        "avg_price": sum(prices) / len(prices),
        "total_value": total_val,
        "max_price": max(prices)
    }

# Глобальна версія для Cache Invalidation
_CACHE_VERSION: int = 1

def invalidate_category_stats_cache() -> None:
    """Інкремент версії та повне очищення кешу при зміні каталогу."""
    global _CACHE_VERSION
    _CACHE_VERSION += 1
    _internal_cached_stats.cache_clear()

@lru_cache(maxsize=128)
def _internal_cached_stats(category: str, version: int, products_tuple: Tuple[Product, ...]) -> Dict[str, Any]:
    filtered = [p for p in products_tuple if p.category == category]
    if not filtered:
        return {"category": category, "count": 0, "avg_price": 0.0, "total_value": 0.0, "max_price": 0.0}
    prices = [p.price for p in filtered]
    total_val = sum(p.price * p.quantity for p in filtered)
    return {
        "category": category,
        "count": len(filtered),
        "avg_price": sum(prices) / len(prices),
        "total_value": total_val,
        "max_price": max(prices)
    }

def calculate_category_stats_cached(products: List[Product], category: str) -> Dict[str, Any]:
    """Кешована версія статистики за категорією з LRU-кешем та підтримкою інвалідації."""
```

```
    return _internal_cached_stats(category, _CACHE_VERSION, tuple(products))
```

Модуль містить математичні алгоритми та аналітичні обчислення. Векторизація NumPy прибирає накладні витрати інтерпретатора Python. Функція heavy_price_adjustment штучно навантажує математичний сопроцесор для симуляції CPU-bound задачі. Для запобігання ситуації повернення застарілих даних впроваджено функцію invalidate_category_stats_cache(), яка змінює глобальну версію та очищає кеш при зміні каталогу товарів.

4.3. parallel.py

Реалізує паралельний та багатопроцесний перерахунок цін, а також потокобезпечне оновлення залишків на складі

```
from __future__ import annotations

import threading

from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

from typing import List

from .analytics import heavy_price_adjustment


def reprice_products_threads(prices: List[float], workers: int = 4) -> List[float]:
    """Паралельний перерахунок цін за допомогою ThreadPoolExecutor."""
    with ThreadPoolExecutor(max_workers=workers) as executor:
        return list(executor.map(heavy_price_adjustment, prices))


def reprice_products_processes(prices: List[float], workers: int = 4, chunksize: int = 250) ->
List[float]:
    """Багатопроцесний перерахунок цін за допомогою ProcessPoolExecutor."""
    with ProcessPoolExecutor(max_workers=workers) as executor:
        return list(executor.map(heavy_price_adjustment, prices, chunksize=chunksize))


class SharedWarehouseStock:
    def __init__(self, initial_stock: int = 0) -> None:
        self.stock = initial_stock
        self.lock = threading.Lock()

    def update_without_lock(self, delta: int) -> None:
        """Небезпечне оновлення залишків без блокування (Race Condition)."""
        current = self.stock
```

```
        self.stock = current + delta

    def update_with_lock(self, delta: int) -> None:
        """Потокобезпечне оновлення залишків із синхронізацією через Lock."""
        with self.lock:
            self.stock += delta

def concurrent_stock_update_race(threads_count: int = 10, operations_per_thread: int = 1000) ->
int:
    warehouse = SharedWarehouseStock(0)
    threads = []
    def worker():
        for _ in range(operations_per_thread):
            warehouse.update_without_lock(1)
    for _ in range(threads_count):
        t = threading.Thread(target=worker)
        threads.append(t)
        t.start()
    for t in threads:
        t.join()
    return warehouse.stock

def concurrent_stock_update_locked(threads_count: int = 10, operations_per_thread: int = 1000) ->
int:
    warehouse = SharedWarehouseStock(0)
    threads = []
    def worker():
        for _ in range(operations_per_thread):
            warehouse.update_with_lock(1)
    for _ in range(threads_count):
        t = threading.Thread(target=worker)
        threads.append(t)
        t.start()
```

```
    for t in threads:
        t.join()
    return warehouse.stock
```

Модуль інкапсулює роботу з пулами потоків і процесів з бібліотеки concurrent.futures. Клас SharedWarehouseStock демонструє усунення гонитви потоків (Race Condition) за допомогою об'єкта синхронізації threading.Lock.

4.4. profiling.py та test_store_manager.py

Модуль реалізує інструменти діагностики продуктивності системи: функцію profile_cpu_and_bottlenecks() для виявлення вузьких місць за допомогою вбудованого модуля cProfile та функцію profile_memory_tracemalloc() для моніторингу виділення динамічної пам'яті через модуль tracemalloc.

```
import cProfile
import pstats
import tracemalloc
import numpy as np
from .repository import ProductRepository
from .analytics import (
    calculate_total_stock_value_baseline,
    reprice_products_baseline,
)


def profile_cpu_and_bottlenecks():
    print("=== ПРОФІЛЮВАННЯ ЦИКЛІВ ТА ВИЗНАЧЕННЯ BOTTLENECK (cProfile) ===")
    products = ProductRepository.generate_sample_data(count=100_000)
    prices = [p.price for p in products[:10_000]]

    profiler = cProfile.Profile()
    profiler.enable()
    calculate_total_stock_value_baseline(products)
    reprice_products_baseline(prices)
    profiler.disable()

    stats = pstats.Stats(profiler)
    stats.sort_stats("cumulative")
```

```
    stats.print_stats(10)


def profile_memory_tracemalloc():
    print("\n=== АНАЛІЗ ВИКОРИСТАННЯ ПАМ'ЯТІ (tracemalloc) ===")
    tracemalloc.start()
    products = ProductRepository.generate_sample_data(count=100_000)
    _, peak_1 = tracemalloc.get_traced_memory()
    print(f"[Python Objects] Кількість: 100k | Пікова пам'ять: {peak_1 / (1024*1024):.2f} MB")


    prices = np.array([p.price for p in products], dtype=np.float64)
    quantities = np.array([p.quantity for p in products], dtype=np.int32)
    _, peak_2 = tracemalloc.get_traced_memory()
    print(f"[NumPy Arrays]   Після створення масивів | Пікова пам'ять: {peak_2 / (1024*1024):.2f}
MB")
    tracemalloc.stop()


if __name__ == "__main__":
    profile_cpu_and_bottlenecks()
    profile_memory_tracemalloc()
```

Модуль відповідає за аналітику витрат системних ресурсів. Використання cProfile дозволяє детерміновано оцінити час виконання функцій програми та кількість їхніх внутрішніх викликів, впорядкувавши результат за параметром cumtime (накопичений час) для локалізації головного bottleneck. Функція на базі tracemalloc фіксує піковий обсяг пам'яті (peak memory), виділений під час створення великої вибірки стандартних об'єктів Python та їхнього векторизованого представлення у масивах NumPy.

test_store_manager.py

У модулі реалізовано автоматизовані тести на базі фреймворків unittest та pytest для верифікації правильності розрахунків оптимізованих версій, коректності кешування та надійності потокобезпечної синхронізації.

```
import unittest
import numpy as np
from src.store_manager.models import Product
from src.store_manager.analytics import (
    calculate_total_stock_value_baseline,
    calculate_total_stock_value_numpy,
```

```
    calculate_category_stats_cached,
    invalidate_category_stats_cache,
    reprice_products_baseline,
    _internal_cached_stats,
)
from src.store_manager.parallel import (
    reprice_products_threads,
    reprice_products_processes,
    concurrent_stock_update_locked,
)

class TestStoreManager(unittest.TestCase):
    def setUp(self):
        self.sample_products = [
            Product(id=1, name="Смартфон", category="Електроніка", price=1000.0, quantity=5),
            Product(id=2, name="Ноутбук", category="Електроніка", price=2500.0, quantity=2),
            Product(id=3, name="Футболка", category="Одяг та взуття", price=50.0, quantity=20),
        ]

    def test_stock_value_consistency(self):
        baseline_val = calculate_total_stock_value_baseline(self.sample_products)
        prices = np.array([p.price for p in self.sample_products], dtype=np.float64)
        quantities = np.array([p.quantity for p in self.sample_products], dtype=np.int32)
        numpy_val = calculate_total_stock_value_numpy(prices, quantities)
        self.assertAlmostEqual(baseline_val, numpy_val, places=2)
        self.assertEqual(baseline_val, 1000.0*5 + 2500.0*2 + 50.0*20)

    def test_parallel_reprice_correctness(self):
        prices = [100.0, 250.0, 500.0]
        sequential = reprice_products_baseline(prices)
        threaded = reprice_products_threads(prices, workers=2)
        multiprocessed = reprice_products_processes(prices, workers=2)
```

```
        self.assertEqual(sequential, threaded)
        self.assertEqual(sequential, multiprocessed)

    def test_caching_and_invalidation(self):
        invalidate_category_stats_cache()
        # Перший виклик - miss
        res1 = calculate_category_stats_cached(self.sample_products, "Електроніка")
        info1 = _internal_cached_stats.cache_info()
        self.assertEqual(info1.misses, 1)

        # Другий виклик - hit
        res2 = calculate_category_stats_cached(self.sample_products, "Електроніка")
        info2 = _internal_cached_stats.cache_info()
        self.assertEqual(info2.hits, 1)
        self.assertEqual(res1, res2)

        # Інвалідація
        invalidate_category_stats_cache()
        res3 = calculate_category_stats_cached(self.sample_products, "Електроніка")
        info3 = _internal_cached_stats.cache_info()
        self.assertEqual(info3.currsize, 1)

    def test_synchronization_lock(self):
        threads = 10
        ops = 500
        expected = threads * ops
        locked_result = concurrent_stock_update_locked(threads_count=threads,
operations_per_thread=ops)
        self.assertEqual(locked_result, expected)

if __name__ == "__main__":
    unittest.main()
```

Тестовий набір гарантує, що застосовані техніки оптимізації не порушують вихідної бізнес-логіки та семантики програми. Тест test_stock_value_consistency перевіряє числову точність векторизованої операції NumPy у порівнянні зі звичайним циклом, test_parallel_reprice_correctness контролює детермінованість багатопотокового та багатопроцесного виконання, test_caching_and_invalidation аналізує статистику попадань (hits) і промахів (misses) кешу та очищення кешованого стану, а test_synchronization_lock підтверджує захист від втрати оновлень (Race Condition) за допомогою об'єкта threading.Lock.

5. Файл pyproject.toml

Файл визначає build system setuptools, метадані проєкту, версію Python 3.10+, залежність від numpy та тестову залежність pytest.

```
[build-system]

requires = ["setuptools>=61.0"]

build-backend = "setuptools.build_meta"


[project]

name = "store-manager"

version = "0.1.0"

description = "Система управління інтернет-магазином (Варіант 3, Лабораторна №9)"

readme = "README.md"

requires-python = ">=3.10"

dependencies = [

  "numpy>=1.24.0",

]


[project.optional-dependencies]

dev = [

  "pytest>=7.0.0",

]


[tool.setuptools.packages.find]

where = ["src"]
```

Центральний декларативний конфігураційний файл проєкту згідно з PEP 518/621. Блок [tool.setuptools.packages.find] налаштовує систему збирання на каталог src.

6. Файл .gitignore

З Git виключено віртуальне середовище, байт-код інтерпретатора, кеші тестування, звіти та тимчасові файли:

@"

__pycache__/

*.py[cod]

*$py.class

.venv/

venv/

*.egg-info/

dist/

build/

.pytest_cache/

"@ | Out-File -Encoding utf8 .gitignore

7. Фрагмент README.md

Онлайн-магазин (Online Store Manager) — Варіант 3

Лабораторна робота №9: Паралельність, багатопроцесність та оптимізація.

Вимоги та встановлення

- Python 3.10+

- Встановлення: pip install -e .

Запуск

- Тести: pytest tests/

- Профілювання: python -m src.store_manager.profiling

- Бенчмарк: python benchmarks/benchmark.py

Документ містить стислий опис функціоналу варіанта №3, правила розгортання проєкту та команди для запуску тестів, профілювання та бенчмарків.

8. Запуск та результат роботи програми

8.1. Результати профілювання (cProfile та tracemalloc)

=== ПРОФІЛЮВАННЯ ЦИКЛІВ ТА ВИЗНАЧЕННЯ BOTTLENECK (cProfile) ===

3220003 function calls in 2.312 seconds

Ordered by: cumulative time

ncalls tottime percall cumtime percall filename:lineno(function)

1 0.007 0.007 2.307 2.307 D:\Робочий стіл\проф. python\online_store_manager\src\store_manager\analytics.py:26(reprice_products_baseline)

10000 1.445 0.000 2.300 0.000 D:\Робочий стіл\проф. python\online_store_manager\src\store_manager\analytics.py:19(heavy_price_adjustment)

800000 0.228 0.000 0.228 0.000 {built-in method math.sin}

800000 0.224 0.000 0.224 0.000 {built-in method math.sqrt}

800000 0.217 0.000 0.217 0.000 {built-in method math.cos}

800000 0.163 0.000 0.163 0.000 {built-in method builtins.abs}

10000 0.022 0.000 0.022 0.000 {built-in method builtins.round}

1 0.005 0.005 0.005 0.005 D:\Робочий стіл\проф. python\online_store_manager\src\store_manager\analytics.py:8(calculate_total_stock_value_baseline)

1 0.000 0.000 0.000 0.000 {method 'disable' of '_lsprof.Profiler' objects}

=== АНАЛІЗ ВИКОРИСТАННЯ ПАМ'ЯТІ (tracemalloc) ===

[Python Objects] Кількість: 100k | Пікова пам'ять: 20.57 MB

[NumPy Arrays] Після створення масивів | Пікова пам'ять: 22.48 MB

Профілювання чітко локалізувало bottleneck — функція heavy_price_adjustment зайняла **99.48%** (2.300 с із 2.312 с) загального часу через масові виклики математичних операцій, що підтверджує CPU-bound характер задачі. Аналіз пам'яті зафіксував пікове навантаження на рівні 20.57 MB для 100 000 об'єктів.

8.2. Результати бенчмарків (benchmark.py)

==========

БЕНЧМАРК ОПТИМІЗАЦІЇ ТА ПАРАЛЕЛЬНОСТІ (ВАРІАНТ 3: ІНТЕРНЕТ-МАГАЗИН)

==========

[Експеримент 1] Розрахунок загальної вартості складу (Stock Value)

Dataset: 10000 | Python: 0.001489s | NumPy: 0.000312s | Speedup: 4.77x

Dataset: 50000 | Python: 0.003825s | NumPy: 0.000131s | Speedup: 29.18x

Dataset: 100000 | Python: 0.019233s | NumPy: 0.000822s | Speedup: 23.41x

[Експеримент 2] Динамічний перерахунок цін (CPU-bound workload, 10,000 позицій)

Sequential Baseline: 0.6771s

Workers: 1 | Threads: 0.7180s (Sp: 0.94x) | Processes: 0.8642s (Sp: 0.78x)

Workers: 2 | Threads: 0.8130s (Sp: 0.83x) | Processes: 0.7301s (Sp: 0.93x)

Workers: 4 | Threads: 0.7465s (Sp: 0.91x) | Processes: 0.7480s (Sp: 0.91x)

Workers: 8 | Threads: 0.6900s (Sp: 0.98x) | Processes: 1.2046s (Sp: 0.56x)

[Експеримент 3] Кешування статистики категорій (LRU Cache)

Cold Cache (Miss): 0.046147s | Warm Cache (Hit): 0.03331000s | Speedup: 1.4x

[Успішно] Результати збережено у benchmarks/results/benchmark_results.csv

Експерименти засвідчили максимальне прискорення від векторизації **NumPy** (до **29.18x**). Для CPU-bound задачі потоки ThreadPoolExecutor виявилися повільнішими за baseline через блокування інтерпретатора GIL, а ProcessPoolExecutor показав накладні витрати міжпроцесної взаємодії (IPC) та старту інтерпретатора у Windows.

8.3. Результати автоматизованого тестування (pytest)

========== test session starts ==========

platform win32 -- Python 3.13.0, pytest-9.2.0, pluggy-1.7.0

rootdir: D:\Робочий стіл\проф. python\online_store_manager

configfile: pyproject.toml

collected 4 items

tests\test_store_manager.py .... [100%]

========== 4 passed in 0.55s ==========

Усі 4 модульні тести успішно підтвердили цілісність логіки: ідентичність результатів між циклами та NumPy, коректність паралельного перерахунку, роботу lru_cache та синхронізацію спільних ресурсів за допомогою Lock.

9. Робота з Git

У проєкті створено три логічні коміти відповідно до стадій реалізації:

Коміт 1: "feat: setup project structure, product models and repository"

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store_manager (master)

$ git add pyproject.toml README.md .gitignore src/store_manager/models.py src/store_manager/repository.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store_manager (master)

$ git commit -m "feat: setup project structure, product models and repository"

[master (root-commit) 1b78a78] feat: setup project structure, product models and repository

5 files changed, 89 insertions(+)

create mode 100644 .gitignore

create mode 100644 README.md

create mode 100644 pyproject.toml

create mode 100644 src/store_manager/models.py

create mode 100644 src/store_manager/repository.py

Ініціалізовано репозиторій, налаштовано файли збирання, базові правила ігнорування артефактів, створено сутність Product та клас репозиторію з генератором вибірок.

Коміт 2: "feat: implement NumPy vectorization, parallel executors and Lock synchronization"

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store_manager (master)

$ git add src/store_manager/analytics.py src/store_manager/parallel.py src/store_manager/__init__.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store_manager (master)

$ git commit -m "feat: implement NumPy vectorization, parallel executors and Lock synchronization"

[master 396be42] feat: implement NumPy vectorization, parallel executors and Lock synchronization

3 files changed, 163 insertions(+)

create mode 100644 src/store_manager/__init__.py

create mode 100644 src/store_manager/analytics.py

create mode 100644 src/store_manager/parallel.py

Створено модуль оптимізованої аналітики, додано векторизацію через NumPy, паралельні пули потоків і процесів, а також потокобезпечний складський лічильник із threading.Lock.

Коміт 3: "test: add test suite, profiling tools and benchmark framework"

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store_manager (master)

$ git add tests/ src/store_manager/profiling.py benchmarks/

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store_manager (master)

$ git commit -m "test: add test suite, profiling tools and benchmark framework"

[master 9650244] test: add test suite, profiling tools and benchmark framework

4 files changed, 301 insertions(+)

create mode 100644 benchmarks/benchmark.py

create mode 100644 benchmarks/results/benchmark_results.csv

create mode 100644 src/store_manager/profiling.py

create mode 100644 tests/test_store_manager.py

Додано повне модульне тестування на базі unittest/pytest, скрипти профілювання CPU та пам'яті, стенд бенчмарку та автоекспорт метрик у CSV.

9.1. Результат git status

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store_manager (master)

$ git status

On branch master

nothing to commit, working tree clean

Команда підтверджує актуальний стан робочого каталогу: всі робочі файли зафіксовані в комітах, незафіксованих змін немає, а тимчасові файли та звіти успішно ігноруються.

9.2. Результат git log –oneline

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store_manager (master)

$ git log --oneline

9650244 (**HEAD** -> **master**) test: add test suite, profiling tools and benchmark framework

396be42 feat: implement NumPy vectorization, parallel executors and Lock synchronization

1b78a78 feat: setup project structure, product models and repository

Лог відображає послідовну та структуровану історію розробки із трьох змістовних етапів: від конфігурації каркаса до оптимізацій та бенчмаркінгу

10.Висновок

У лабораторній роботі досліджено методи оптимізації Python-застосунку для інтернет-магазину (Варіант 3). За допомогою cProfile локалізовано головний bottleneck — функцію ціноутворення heavy_price_adjustment (99.48% часу), а tracemalloc зафіксував 20.57 MB пікової пам'яті. Векторизація NumPy забезпечила найбільше прискорення (до 29.18x) при розрахунку вартості залишків. Для CPU-bound задачі використання ThreadPoolExecutor

виявилося неефективним через GIL, а ProcessPoolExecutor зазнав накладних витрат на старт процесів у Windows. Стан гонитви (Race Condition) успішно усунуто через threading.Lock, а повторні розрахунки прискорено за допомогою LRU-кешу з інвалідацією. Експерименти довели, що паралелізм не дає автоматичного прискорення без урахування типу навантаження.